

Monkey Patching

Monkey Patching is the process of modifying or adding variables and methods to a class or an object at runtime (while the program is executing) without changing the original source code.

In simple words, Monkey Patching means changing the behavior of an existing class or object during runtime.

Real-Life Example

Imagine your college has already printed your ID card.

Later, the administration sticks a new barcode sticker on the ID card instead of printing a completely new one.

The original ID card is the same, but its behavior/functionality has changed. This is similar to Monkey Patching.

Why is it called "Monkey Patching"?

The term "Monkey Patching" comes from the idea of making dynamic, unexpected, or playful changes to an existing program while it is running.

- Monkey refers to making changes in a flexible or playful way.
 - Patching means modifying or fixing existing code.
-

Why do we use Monkey Patching?

Monkey patching is mainly used to:

- Fix bugs in third-party libraries.
 - Add new functionality without modifying the original code.
 - Replace methods for testing (mocking).
 - Customize library behaviour temporarily.
-

Example 1: Replacing an Existing Method

```
class Student:
    def study(self):
        print("Student is studying Python")

def new_study(self):
    print("Student is studying Java")

Student.study = new_study

s = Student()
s.study()
```

Output: Student is studying Java

Example 2: Adding a New Method

```
class Student:
    def study(self):
        print("Studying")

def play(self):
    print("Playing Cricket")

Student.play = play

s = Student()

s.study()
s.play()
```

Output

Studying
Playing Cricket

Originally, the Student class had only one method.

Using monkey patching, we added another method without modifying the original class.

Example 3: Monkey Patching an Object

```
class Student:
    def study(self):
        print("Studying")

def sleep(self):
    print("Sleeping")

s = Student()

s.sleep = sleep

s.sleep(s)
```

Output: Sleeping

Only this object (s) gets the new method. Other Student objects will not have it. Hence we need to pass self (object) as an argument while calling the method.

Example 4: Proper Object Monkey Patching (MethodType)

MethodType is a class from Python's built-in types module that binds a function to an **object**, making the function behave like an instance method.

Without MethodType, a function attached to an object is just a normal function. With MethodType, Python automatically passes the object as self.

Syntax

```
from types import MethodType
object.method = MethodType(function, object)
```

```
from types import MethodType
```

```
class Student:
    pass

def study(self):
    print("Studying")

s1 = Student()

s1.study = MethodType(study, s1)

s1.study()
```

Output: Studying

Example 4: Changing a Class Variable

```
class Student:
    college = "ABC College"

print(Student.college)

Student.college = "XYZ College"

print(Student.college)
```

Output

ABC College
XYZ College
The class variable was modified during runtime.

Example 5: Adding an Instance Variable at Runtime

```
class Student:

    def display(self):
        print("Student Class")

s1 = Student()
s1.name = "Rahul"
print(s1.name)
```

Output: Rahul

Example 6: Modifying an Existing Instance Variable

```
class Student:
    def __init__(self):
        self.name = "Rahul"

s1 = Student()
print(s1.name)

s1.name = "Anjali"
print(s1.name)
```

Output

Rahul
Anjali
Here, the value of the instance variable is changed at runtime.
